

CONFIDENTIAL

AI Sales Training Platform

Technical Documentation

Frontend Developer Guide & Architecture Reference

Platform	AI Sales Training Platform
Framework	Next.js 15 · React 19 · TypeScript
State	Redux Toolkit · RTK Query · Redux Persist
Real-time	WebSocket · MediaRecorder · Web Audio API
Integrations	Stripe · Google OAuth · Recharts
Version	1.0.0
Status	Production Ready

© 2025 AI Sales Training Platform · Confidential

AI Sales Training Platform — Frontend Technical Documentation

Confidential · © 2025 AI Sales Training Platform

Field	Value
Platform	AI Sales Training Platform
Framework	Next.js 15 (App Router) · React 19 · TypeScript
State	Redux Toolkit · RTK Query · Redux Persist
Styling	Tailwind CSS 4 · Radix UI (shadcn)
Real-time	Native WebSocket · MediaRecorder · Web Audio API
Payments	Stripe Elements
Auth	JWT (cookies) · Google OAuth
Version	1.0.0
Status	Production Ready

Table of Contents

- 1. [Project Overview](#project-overview)
- 2. [Tech Stack](#tech-stack)
- 3. [Architecture](#architecture)
- 4. [Installation & Setup](#installation--setup)
- 5. [Environment Variables](#environment-variables)
- 6. [Running the Application](#running-the-application)
- 7. [Route Reference](#route-reference)
- 8. [Authentication](#authentication)
- 9. [API Integration (RTK Query)](#api-integration-rtk-query)
- 10. [AI Service Integration](#ai-service-integration)
- 11. [WebSocket — Live Conversation](#websocket--live-conversation)
- 12. [Audio Pipeline](#audio-pipeline)
- 13. [Meeting Creation Flow](#meeting-creation-flow)
- 14. [State Management](#state-management)
- 15. [TypeScript Interfaces](#typescript-interfaces)
- 16. [Component Reference](#component-reference)
- 17. [Enums & Valid Values](#enums--valid-values)
- 18. [Flow — How It All Works](#flow--how-it-all-works)
- 19. [Error Handling](#error-handling)

20. [Security Notes](#security-notes)

Project Overview

The frontend is a Next.js 15 application for an AI-powered sales training platform. Salespeople configure practice meetings through a 5-step wizard, then conduct live voice conversations with AI company representatives over WebSocket. Post-meeting analytics, account management, subscriptions, and AI insights are available from the dashboard.

Key Features

- 5-step meeting wizard (product → company → participants → objectives → live call)
 - Real-time voice conversation via WebSocket with AI representatives
 - Animated avatar lip-sync during AI speech playback
 - Auto silence detection (volume-based VAD) for hands-free turn-taking
 - Post-meeting insights: talk ratio, questions asked, topics, risks & opportunities
 - My Account view with company data, meeting history, and AI insights
 - Subscription management with Stripe payment integration
 - Google OAuth sign-in and email/password authentication
 - JWT auto-refresh on 401 responses
 - Public landing page with pricing, FAQ, and contact forms
-

Tech Stack

Layer	Technology
Framework	Next.js 15.5 (App Router)
UI Library	React 19
Language	TypeScript 5
State	Redux Toolkit + RTK Query
Persistence	Redux Persist (localStorage)
Styling	Tailwind CSS 4
UI Components	Radix UI / shadcn-style primitives
Forms	React Hook Form + Zod
Charts	Recharts
Animations	Framer Motion
Toasts	Sonner
Auth	js-cookie + jwt-decode
OAuth	@react-oauth/google
Payments	@stripe/react-stripe-js
Real-time	Native WebSocket (not socket.io)
Audio	MediaRecorder + Web Audio API

Architecture

```
Browser (Next.js Client)
├─ App Router Pages (public / auth / dashboard)
├─ Redux Store
│  ├─ userSlice (persisted)
│  ├─ startMeetingSlice (persisted)
│  └─ baseApi (RTK Query – not persisted)
├─ Cookies (token, refreshToken, meeting state)
└─ Step5 LiveConversation
   ├─ POST AI /api/meeting/:id/start
   ├─ WebSocket wss://ai-julientmts.../live-conversation/:id
   ├─ MediaRecorder → audio_chunk events
   └─ AudioContext → TTS playback + lip-sync

Backend API (RTK Query)
└─ https://api-julientmts.aiteamtwo.com/api/v1

AI Service (direct fetch + WebSocket)
└─ https://ai-julientmts.aiteamtwo.com
```

Directory Structure

```
src/
├─ app/                # Next.js App Router
│  ├─ layout.tsx       # Root: Redux, Google OAuth, Toaster
│  ├─ (commonLayout)/  # Public pages (/ , /about, /pricing, /contact)
│  ├─ (auth)/          # Auth pages (/signIn, /signUp, /otp, etc.)
│  └─ (dashboard)/     # Protected dashboard (/dashboard/*)
├─ components/         # Feature & UI components
│  ├─ startNewMeeting/ # Step1-5 wizard + live conversation
│  ├─ landingPage/     # Marketing sections
│  ├─ dashboard/       # Stats, charts, subscription
│  ├─ insights/        # Post-meeting analytics
│  ├─ myAccount/       # Account detail views
│  ├─ shared/          # Navbar, Footer, buttons
│  └─ ui/              # shadcn primitives
├─ feature/auth/       # SignIn, SignUp, OTP, password flows
├─ hooks/              # useGetMe, use-mobile
├─ interfaces/global.ts # Core TypeScript types
├─ middleware.ts        # Dashboard JWT protection
├─ redux/
│  ├─ api/             # RTK Query slices
│  ├─ features/        # Redux slices
│  ├─ store.ts
│  └─ Provider.tsx
└─ utils/              # Date/time helpers
```

Installation & Setup

Prerequisites

- Node.js 18+
- npm, yarn, pnpm, or bun
- Google OAuth Client ID (for Google sign-in)
- Stripe Publishable Key (for subscriptions)

Install

```
npm install
```

Copy environment variables (see below) into `.env` at project root.

Environment Variables

Variable	Required	Used In	Description
<code>NEXT_PUBLIC_GOOGLE_CLIENT_ID</code>	Yes	<code>app/layout.tsx</code>	Google OAuth client ID
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>	Yes	<code>PassPayment.tsx</code>	Stripe publishable key
<code>NEXT_PUBLIC_API_URL</code>	No*	—	Defined in <code>.env</code> but not used — API URL is hardcoded
<code>NEXT_PUBLIC_DOMAIN_URL_ONE</code>	No	—	Not referenced in source

* **Note:** `baseApi.ts` hardcodes `https://api-julientmts.aiteamtwo.com/api/v1`. AI service URLs are also hardcoded in `Step5.tsx`.

Running the Application

```
npm run dev    # Development server on port 3054
npm run build  # Production build
npm run start  # Production server on port 3054
npm run lint   # ESLint
```

- **Local URL:** `http://localhost:3054`
- **Dashboard:** `http://localhost:3054/dashboard/home` (requires login)

Route Reference

Public Routes — `(commonLayout)`

URL	Page	Description
<code>/</code>	Landing page	Hero, features, testimonials, FAQ
<code>/about</code>	About us	Company story and stats
<code>/pricing</code>	Pricing	Subscription plans
<code>/contact</code>	Contact	Public contact form

Auth Routes — (auth)

URL	Component	Description
/signIn	SignIn.tsx	Email/password + Google login
/signUp	SignUp.tsx	Registration
/otp	Otp.tsx	Email OTP verification
/forget-password	ForgetPass.tsx	Request password reset
/verify-forget-password	FpOtpVerification.tsx	OTP for reset flow
/reset-password	ResetPassword.tsx	Set new password

Dashboard Routes — (dashboard) (protected by middleware)

URL	Description
/dashboard/home	Main dashboard — stats, recent meetings, AI insights
/dashboard/home/insights	Post-meeting analytics (?meetingId=&sessionId=)
/dashboard/home/viewSummary	Meeting transcript summary
/dashboard/home/replay	Meeting audio replay
/dashboard/home/totalMeeting	Total meetings list
/dashboard/home/questionsAsk	Questions asked stats
/dashboard/startNewMeeting	5-step meeting wizard (?step=1-5)
/dashboard/startNewMeeting/startAiMeeting	UI mock (no real AI)
/dashboard/myAccount	Company account list
/dashboard/myAccount/[accountDetailsId]	Single account detail
/dashboard/subscriptions	Subscription plans & payment
/dashboard/settingPage	Profile & password settings
/dashboard/help	Help FAQ & support form
/dashboard/customPrompt	Custom prompt (sidebar hidden)

Query Parameters

Param	Used On	Purpose
?step=1-5	/dashboard/startNewMeeting	Wizard step navigation
?id=<companyId>	/dashboard/startNewMeeting?step=3	Skip to participants from account
?meetingId=&sessionId=	Insights, summary, replay	Target specific session
?redirect=	/signIn	Post-login redirect (default: /dashboard/home)

Authentication

Token Storage

Storage	Key	Content
Cookie	<code>token</code>	JWT access token
Cookie	<code>refreshToken</code>	JWT refresh token
Redux	<code>user</code> slice	<code>user</code> , <code>token</code> , <code>refreshToken</code>
localStorage	Redux Persist	<code>user</code> + <code>startMeeting</code> slices

Sign-In Flow

1. User submits credentials → `POST /auth/login` or `POST /auth/google`
2. On success: set cookies, `dispatch(setUser(...))`
3. Only `role === "USER"` allowed on dashboard
4. Redirect to `?redirect=` or `/dashboard/home`

Middleware (`src/middleware.ts`)

- **Matcher:** `/dashboard/:path*` only
- No `token` AND no `refreshToken` → redirect `/signIn`
- `refreshToken` only → allow through (client-side refresh)
- JWT decode failure + no refresh → redirect `/signIn`
- `role !== "USER"` → redirect `/signIn`

Auto Token Refresh (`baseApi.ts`)

On any HTTP 401:

1. Read `refreshToken` cookie
2. `POST /auth/refresh-token` with `{ refreshToken }` (no auth header)
3. On success: set new `token` cookie, retry original request
4. On failure: clear cookies via `logout()`

Logout

1. `POST /auth/logout` with refresh token
2. Remove cookies, `dispatch(logoutFc())` , `localStorage.clear()`
3. Redirect to `/`

API Integration (RTK Query)

Base URL: `https://api-julientmts.aiteamtwo.com/api/v1`

Auth Header: `Authorization: Bearer <token>`

Tag Types: `User` , `Meeting` , `Subscription` , `MyAccount` , `Notification`

Standard response shape:

```
{
  "success": true,
  "data": { ... },
  "message": "...
}
```

Auth API — `/auth/` and `/users/`

Hook	Method	Endpoint
<code>useSignUpMutation</code>	POST	<code>/auth/signup</code>
<code>useVerifyEmailMutation</code>	POST	<code>/auth/verify-otp</code>
<code>useResendCodeMutation</code>	POST	<code>/auth/resend-otp/signup</code>
<code>useSignInMutation</code>	POST	<code>/auth/login</code>
<code>useForgetPasswordMutation</code>	POST	<code>/auth/forgot-password</code>
<code>useVerifyForgetPasswordOtpMutation</code>	POST	<code>/auth/verify-forgot-password-otp</code>
<code>useResendForgetPasswordVeirifyOtpMutation</code>	POST	<code>/auth/resend-otp/forgot-password</code>
<code>useResetPasswordMutation</code>	POST	<code>/auth/reset-password</code>
<code>useGoogleSignInMutation</code>	POST	<code>/auth/google</code>
<code>useLogoutMutation</code>	POST	<code>/auth/logout</code>
<code>useUpdateProfileMutation</code>	PATCH	<code>/users/profile</code>
<code>useChangePasswordMutation</code>	POST	<code>/auth/change-password</code>
<code>useContactSupportMutation</code>	POST	<code>/support/contact</code>

User / Notifications API

Hook	Method	Endpoint
<code>useGetMeQuery</code>	GET	<code>/users/profile</code>
<code>useGetAllNotificationsQuery</code>	GET	<code>/notifications</code>
<code>useReadNotificationMutation</code>	PATCH	<code>/notifications/:id/read</code>

Dashboard API

Hook	Method	Endpoint
<code>useGetUserDashboardStatsQuery</code>	GET	<code>/dashboard/user/stats</code>

Meeting API

Hook	Method	Endpoint	Description
useMeetingSalesPersonMutation	POST	/meeting/salesperson	Create salesperson (FormData)
useMeetngCompanyMutation	POST	/meeting/company	Create company from URL
useMeetingCompanyRepresentitiveMutation	POST	/meeting/company/:companyId/representatives	Add representatives
useCreateMeetingIdMutation	POST	/meeting/create	Create meeting + top 5 questions
useUpdateMeetingMutation	PATCH	/meeting/:meetingId/status	Update meeting status

Salesperson FormData fields:

Field	Type	Description
bodyData	JSON string	{ product_name, description, product_url }
materials	file[]	PDF, PPTX, DOC, JPG, PNG

Create Meeting payload:

```
{
  "salesperson_id": "abc123",
  "company_id": "xyz789",
  "meeting_mode": "1-on-1",
  "representatives": ["rep1"],
  "meeting_goal": "Discovery",
  "personality": "nice",
  "duration_minutes": 30,
  "difficulty": "intermediate",
  "sales_methodology": "MEDDIC",
  "methodology_description": "",
  "status": "pending"
}
```

My Account API

Hook	Method	Endpoint
useMyAccountListQuery	GET	/meeting/company
useDeleteAccountMutation	DELETE	/meeting/company/:id
useSingleAccountDetailsQuery	GET	/meeting/company/:id/account-details
useConversationHistoryQuery	GET	/meeting/:meeting_id/history?session_id=
useConversationInsightsQuery	GET	/meeting/:meeting_id/analytics?session_id=

Subscription API

Hook	Method	Endpoint
<code>useGetAllSubscriptionsQuery</code>	GET	<code>/plans?interval=</code>
<code>useGetSinglePlanQuery</code>	GET	<code>/plans/:id</code>
<code>usePaymentMethodMutation</code>	POST	<code>/payment_methods</code>
<code>useSubscriptionMutation</code>	POST	<code>/subscriptions</code>
<code>useActiveSubscriptionQuery</code>	GET	<code>/subscriptions/me</code>
<code>useCancelSubscriptionMutation</code>	PATCH	<code>/subscriptions/cancel</code>

Landing Page API

Hook	Method	Endpoint
<code>usePostSupportContactMutation</code>	POST	<code>/support/contact/public</code>
<code>usePostNewsletterSubscribeMutation</code>	POST	<code>/newsletter/subscribe</code>

AI Service Integration

The live meeting (Step 5) communicates directly with the AI backend — **not** through RTK Query.

Action	Method	URL
Start meeting	POST	<code>https://ai-julientmts.aiteamtwo.com/api/meeting/:meetingId/start</code>
End meeting	POST	<code>https://ai-julientmts.aiteamtwo.com/api/meeting/:meetingId/end</code>
Live conversation	WebSocket	<code>wss://ai-julientmts.aiteamtwo.com/conversations/api/conversation/ws/live-conversation/:meetingId</code>

Note: These calls use plain `fetch` / `WebSocket` without JWT auth headers.

WebSocket — Live Conversation

Component: `src/components/startNewMeeting/Step5.tsx`

Connection Lifecycle

1. User clicks Start → 5-4-3-2-1-GO countdown overlay
2. After 3 seconds → `connectToMeeting()`
3. `POST .../api/meeting/:meetingId/start`
4. Open WebSocket to AI conversation service
5. On `connected` → display reps, start timer, auto `startListening()`
6. Heartbeat: `{ type: "ping" }` every 25 seconds

7. On disconnect/end → send `{ type: "disconnect" }`, close WS, `POST .../end`, `PATCH /meeting/:id/status` with `{ status: "completed" }`

Client → Server Messages

type	Payload	Description
audio_chunk	<code>{ data: base64, is_speaking: true/false }</code>	Microphone audio chunk
ping	—	Keepalive heartbeat
disconnect	—	End session gracefully

Send `is_speaking: false` on the last chunk to signal end of user speech.

Server → Client Messages

type	Description
connected	Connection established — includes reps, duration, difficulty
transcription	User speech transcribed — live-update transcript bubble
ai_thinking	AI generating response — mic disabled
ai_response_text	Streaming AI text (primary/secondary speaker)
ai_audio_complete	Base64 TTS audio ready for playback
error	Error message — resume listening
no_audio	No audio detected — resume listening
pong	Response to ping

connected Event Example

```
{
  "type": "connected",
  "meeting_id": "meet123",
  "session_id": "sess456",
  "duration_minutes": 30,
  "difficulty": "intermediate",
  "representatives": [
    {
      "id": "rep1",
      "name": "John Smith",
      "role": "CTO",
      "personality": ["nice"],
      "is_decision_maker": true
    }
  ]
}
```

Audio Pipeline

```
Microphone
↓ getUserMedia (echoCancellation, noiseSuppression, 48kHz)
MediaRecorder (audio/webm;codecs=opus, 100ms chunks)
↓ FileReader → base64
WebSocket audio_chunk (is_speaking: true)
↓
Volume Monitor (AnalyserNode, polled every 100ms)
↓ 800ms below threshold (SILENCE_THRESHOLD = 10)
WebSocket audio_chunk (is_speaking: false)
↓
Backend: Whisper STT → GPT → ElevenLabs TTS
↓
ai_audio_complete (base64 audio)
↓
AudioContext.decodeAudioData → playback
↓
AnalyserNode → canvas lip-sync (drawMouth)
↓
Queue drains → auto startListening() after 800ms
```

Constants (Step5.tsx)

Constant	Value	Purpose
SILENCE_DELAY_MS	800	Ms of silence before sending end-of-speech
SILENCE_THRESHOLD	10	Avg frequency amplitude threshold
Heartbeat interval	25s	WebSocket ping
Auto-listen delay	800ms	Delay before mic restarts after AI

Mic Constraints

- Disabled while AI is thinking or speaking
- Auto-restarts after AI audio queue finishes
- Denied access → system transcript message shown

Meeting Creation Flow

Orchestrated by `src/app/(dashboard)/dashboard/startNewMeeting/page.tsx`.

UI step order (note: component numbers differ from UI order):

```
Step 1 – Step1.tsx: "What Are You Selling?"
→ POST /meeting/salesperson (FormData)
→ Redux: setProductValue
→ Cookie: salesperson_id

Step 2 – Step3.tsx: "Company Information"
→ POST /meeting/company { company_url, salesperson_id }
→ Redux: setCompanyData
→ Display scraped company data

Step 3 – Step2.tsx: "Who Are You Meeting?"
→ POST /meeting/company/:companyId/representatives
→ Redux: setParticipantsValue
→ Cookies: companyId, last_voice_id, participantGenderMap
→ Participant count limited by subscription meetingMode

Step 4 – Step4.tsx: "Meeting Objective"
→ Auto-submits on field change (useEffect)
→ POST /meeting/create → meeting_id + top_5_questions
→ Redux: setMeetingPayload
→ Cookie: meetingId
→ User edits questions, clicks "Next Step"

Step 5 – Step5.tsx: Live AI Conversation
→ Countdown → POST AI /start → WebSocket
→ Voice conversation with AI reps
→ End → POST AI /end + PATCH status completed
```

Shortcut: `/dashboard/startNewMeeting?step=3&id=<accountDetailsId>` pre-sets salesperson and skips to participants.

State Management

Redux Store

```
combineReducers({
  user: userReducer,           // persisted
  startMeeting: startMeetingReducer, // persisted
  baseApi: baseApi.reducer,    // NOT persisted
})
```

userSlice Actions

Action	Effect
<code>setUser</code>	Set <code>user</code> , <code>token</code> , <code>refreshToken</code>
<code>logoutFc</code>	Clear all to <code>null</code>

startMeetingSlice Actions

Action	Effect
setProductValue	Step 1 salesperson/product data
setCompanyData	Step 3 company data
setParticipantsValue	Step 2 representative IDs
setMeetingPayload	Step 4 meeting config + questions

Meeting Cookies (cross-step persistence)

Cookie	Set By	Purpose
salesperson_id	Step1	Salesperson ID
companyId	Step2	Company ID
last_voice_id	Step2	Meeting mode (e.g. 1-on-2)
participantGenderMap	Step2	Avatar gender mapping
meetingId	Step4	Live meeting WebSocket ID

TypeScript Interfaces

File: src/interfaces/global.ts

```
interface UserProfile {
  id: string;
  firstName: string;
  lastName: string;
  email: string;
  profilePicture: string | null;
  role: string;
}

interface ProductValue {
  salesperson_id: string;
  productName: string;
  description: string;
  uploadProduct: File | null;
}

interface CompanyData {
  company_id: string;
  company_size: string | null;
  headquarters: string | null;
  revenue: string | null;
  industry: string | null;
  tech_stack: string[];
  open_positions: string | null;
  founded_year: string | null;
  description: string | null;
  social_links: Record<string, string>;
  data_sources: string[];
}

interface Participant {
  name: string;
  role: string;
  note: string;
  decisionMaker: boolean;
  linkedin: string;
}

interface StartMeetingState {
  product: ProductValue | null;
  companyData: CompanyData | null;
  participants: Participant[];
  payloadData: MeetingPayload | null;
}
```

Component Reference

Landing Page

Component	Purpose
<code>HerroSection.tsx</code>	Hero banner
<code>InfoCounter.tsx</code>	Animated stats
<code>FeatureSection.tsx</code>	Feature cards
<code>BashboardSection.tsx</code>	Dashboard preview
<code>TestimonialSection.tsx</code>	Customer testimonials
<code>Faq.tsx</code>	FAQ accordion
<code>nextMeeting.tsx</code>	CTA section

Dashboard

Component	Purpose
<code>section-cards.tsx</code>	Stats overview cards
<code>recent-mettings.tsx</code>	Recent meetings table
<code>HomeAiInsights.tsx</code>	AI performance insights
<code>StatsCards.tsx</code>	Qualification stats
<code>TalkRatioChart.tsx</code>	Talk ratio visualization
<code>QualificationTrendChart.tsx</code>	Trend chart

Insights (Post-Meeting)

Component	Data Source
<code>ChartBarDefault.tsx</code>	Conversation turns
<code>TalkTimeDistribution.tsx</code>	Talk time percentages
<code>insightsCard.tsx</code>	Listening grade, questions
<code>topicDiscus.tsx</code>	Topics discussed
<code>RisksOpportunities.tsx</code>	Risks & upsell opportunities

My Account

Component	Purpose
<code>fastGrowth.tsx</code>	Company overview
<code>myAccountRecentMeeting.tsx</code>	Meeting history
<code>aiInsights.tsx</code>	Account-level AI insights
<code>OpportunitiesSection.tsx</code>	Upsell opportunities

Subscription

Component	Purpose
<code>Subscription.tsx</code>	Plan listing
<code>PassPayment.tsx</code>	Stripe Elements checkout
<code>LoginRequiredModal.tsx</code>	Auth gate for payment

Layout

Component	Purpose
<code>app-sidebar.tsx</code>	Dashboard navigation + logout
<code>site-header.tsx</code>	Top header bar
<code>Navbar.tsx</code>	Public navigation
<code>Footer.tsx</code>	Public footer

Enums & Valid Values

meeting_mode

Value	Description
<code>"1-on-1"</code>	1 representative
<code>"1-on-2"</code>	2 representatives
<code>"1-on-3"</code>	3 representatives

Set via subscription plan `meetingMode` and stored in `last_voice_id` cookie.

personality

`angry` · `arrogant` · `soft` · `cold_hearted` · `nice` · `cool` · `not_well` · `analytical` · `professional` · `casual` · `direct`

difficulty

`beginner` · `intermediate` · `advanced` · `expert`

sales_methodology

MEDDIC · Challenger Sales · BANT · SPIN Selling · MEDDPIC · Value Selling

meeting duration

5, 10, 15, 20, 25, 30 minutes (selectable in Step 4)

meeting status

pending → active → completed

Flow — How It All Works

```
Step 1 – Register / Sign In
  → POST /auth/signup or /auth/login
  → JWT stored in cookies

Step 2 – Dashboard Home
  → GET /dashboard/user/stats
  → View recent meetings and AI insights

Step 3 – Start New Meeting (Wizard)
  → Step 1: Create salesperson profile
  → Step 2: Scrape company from URL
  → Step 3: Add AI representatives
  → Step 4: Configure meeting + get top 5 questions
  → Step 5: Live voice conversation

Step 4 – Live Conversation (Step 5)
  → Countdown overlay
  → POST AI /meeting/:id/start
  → WebSocket connect
  → Speak → audio_chunk → transcription
  → AI responds → ai_response_text + ai_audio_complete
  → Auto turn-taking until timer ends or user ends

Step 5 – End Meeting
  → POST AI /meeting/:id/end
  → PATCH /meeting/:id/status { status: "completed" }

Step 6 – Review Results
  → /dashboard/home/insights?meetingId=&sessionId=
  → /dashboard/myAccount/[id] for account-level history
```

Error Handling

RTK Query / Mutations

```
try {
  const response = await mutation(payload).unwrap();
  if (response?.success) { /* success */ }
} catch (error: any) {
  const msg = error?.data?.errorMessage?.[0]?.message
    || error?.data?.message
    || "Something went wrong";
  toast.error(msg);
}
```

UI Async States

Component	When Used
Loading	RTK Query <code>isLoading</code>
ErrorState	Full-page error with reload
WithEmptyState	Loading → error → empty → content
Sonner <code>toast</code>	Inline API/form errors

WebSocket Errors

- WS `onerror` → toast "WebSocket connection error"
- `error` message type → system transcript + resume listening
- Mic denied → system message in transcript
- Meeting end API failures → silently caught (UI already cleaned up)

Middleware

Redirect to `/signIn` without toast (server-side).

Security Notes

- Never commit `.env` to version control
- JWT tokens stored in cookies — use `secure: true` and `sameSite: "strict"` on refresh
- Dashboard protected by Next.js middleware (`/dashboard/*`)
- Only `role === "USER"` allowed on dashboard routes
- AI service WebSocket/fetch calls do not send auth headers
- Stripe publishable key is safe for client-side use
- Google OAuth uses `NEXT_PUBLIC_GOOGLE_CLIENT_ID` only (no secret on client)
- Consider wiring `NEXT_PUBLIC_API_URL` env var instead of hardcoded production URL
- `localStorage.clear()` on logout removes all persisted Redux state